

INCOSE SEBoK Core Ontology

Usage Guide and Relationship Map

Version 1.0 | February 2025

Table of Contents

1. Overview
2. Core Concept Hierarchy
3. Key Relationship Patterns
4. Usage Scenarios
5. Integration Patterns
6. SPARQL Query Examples
7. Alignment with Standards

1. Overview

This ontology captures the fundamental concepts from the SEBoK (Systems Engineering Body of Knowledge) v2.13 that are universally applicable across all systems engineering domains. It provides a semantic foundation for:

- System Life Cycle Management: From concept through retirement
- Process Integration: Connecting technical management, definition, realization, and deployment processes
- Artifact Traceability: Linking requirements, architectures, models, and other work products
- Stakeholder Management: Relating people, roles, and responsibilities to systems
- Quality Assurance: Tracking quality attributes and their relationships to systems

Primary Standards Alignment:

- ISO/IEC/IEEE 15288:2023 (System Life Cycle Processes)
- ISO/IEC/IEEE 24748-1:2024 (Life Cycle Management)
- INCOSE Systems Engineering Handbook v5.0

2. Core Concept Hierarchy

2.1 System Concepts

- System (top-level)
 - EngineeredSystem
 - ProductSystem
 - ServiceSystem
 - EnterpriseSystem
 - SystemOfSystems
 - NaturalSystem
 - SocialSystem

Key Properties:

- Every system hasContext (SystemContext)
- Systems are composedOf / partOf other systems
- Systems interactsWith other systems
- Systems dependsOn other systems

2.2 Life Cycle Hierarchy

- LifeCycle
- LifeCycleModel
- LifeCycleStage
 - ConceptStage
 - DevelopmentStage
 - ProductionStage
 - UtilizationStage
 - SupportStage
 - RetirementStage

2.3 Process Hierarchy

- Process
 - LifeCycleProcess
 - TechnicalManagementProcess
 - ProjectPlanning
 - ProjectAssessmentAndControl
 - DecisionManagement
 - RiskManagement
 - ConfigurationManagement
 - InformationManagement
 - RequirementsManagement
 - QualityManagement
 - Measurement
 - SystemDefinitionProcess
 - BusinessOrMissionAnalysis
 - StakeholderNeedsDefinition
 - SystemRequirementsDefinition
 - SystemArchitectureDefinition
 - SystemDetailedDesign
 - SystemRealizationProcess
 - SystemAnalysis
 - SystemImplementation
 - SystemIntegration
 - SystemVerification
 - SystemDeploymentProcess
 - SystemTransition
 - SystemValidation

- SystemOperation
- SystemMaintenance
- SystemDisposal

3. Key Relationship Patterns

3.1 Requirements Flow Pattern

This pattern shows how stakeholder requirements flow through the system definition process:

1. Customer expresses stakeholder requirements
2. Stakeholder requirements derive system requirements
3. System requirements are allocated to subsystems
4. Subsystems satisfy their allocated requirements

Example:

```
SystemX hasStakeholder Customer1
StakeholderReq1 expressedBy Customer1
SystemReq1 derivedFrom StakeholderReq1
SystemReq1 allocatedTo ProcessingSubsystem
ProcessingSubsystem satisfiesRequirement SystemReq1
```

3.2 Process Artifact Flow Pattern

This pattern demonstrates how processes consume and produce artifacts:

1. Process consumes input artifacts
2. Process performs transformations
3. Process produces output artifacts
4. Output artifacts become inputs to subsequent processes

3.3 Life Cycle Progression Pattern

Systems progress through sequential life cycle stages:

ConceptStage → DevelopmentStage → ProductionStage → UtilizationStage → SupportStage → RetirementStage

Each stage has specific processes applied and artifacts produced.

4. Usage Scenarios

4.1 Aerospace Product Development

Context: Developing a new satellite system with complex integration requirements

Key Characteristics:

- Type: ProductSystem
- Life Cycle Model: V-Model
- Development Approach: Sequential Development
- Primary Stakeholders: Space Agency (Customer), Prime Contractor
- Critical Quality Attributes: Reliability, Safety

This scenario emphasizes rigorous verification and validation at each stage, with formal reviews and approvals before progressing.

4.2 Service System Design

Context: Designing an enterprise IT service

Key Characteristics:

- Type: ServiceSystem
- Life Cycle Model: ITIL Model
- Development Approach: Agile Development
- Primary Stakeholders: Business Users, IT Operations
- Critical Quality Attributes: Availability, Usability

This scenario focuses on rapid iteration, continuous feedback, and service-oriented architecture.

4.3 System of Systems Integration

Context: Integrating multiple independent defense systems

Key Characteristics:

- Type: SystemOfSystems
- Components: Radar System, Communication System, Command Center
- Primary Challenge: Interoperability
- Critical Quality Attributes: Interoperability, Security, Resilience

This scenario requires coordination across independently managed systems with distributed governance.

5. Integration Patterns

5.1 Integration with Domain Ontologies

The INCOSE core ontology is designed to be extended by domain-specific ontologies. Domain ontologies should:

- Subclass core concepts (e.g., AircraftSystem subClassOf ProductSystem)
- Specialize relationships (e.g., hasAircraftSubsystem subPropertyOf composedOf)
- Add domain-specific properties while maintaining semantic compatibility
- Preserve transitivity and inverse relationships where inherited

5.2 Integration with Upper Ontologies

The ontology can be mapped to upper ontologies like BFO (Basic Formal Ontology):

- System → MaterialEntity
- Process → Process
- Artifact → InformationContentEntity
- LifeCycleStage → TemporalRegion

This enables integration with broader knowledge management frameworks.

5.3 Integration with Project Management

Project management ontologies can integrate through:

- Project performsProcess (SE Processes)
- WorkPackage producesArtifact (SE Artifacts)
- Deliverable subClassOf Artifact
- Milestone appliedDuringStage (Life Cycle Stages)

6. SPARQL Query Examples

6.1 Find All Requirements for a System

```
PREFIX incose: <https://ontologies.incose.org/sebok/core/>

SELECT ?req ?reqText ?stakeholder
WHERE {
    :MySystem incose:hasRequirement ?req .
    ?req incose:requirementText ?reqText .
    OPTIONAL { ?req incose:expressedBy ?stakeholder }
}
```

6.2 Trace Requirements to Architecture

```
PREFIX incose: <https://ontologies.incose.org/sebok/core/>

SELECT ?req ?arch ?element
WHERE {
    ?req a incose:SystemRequirement ;
        incose:allocatedTo ?element .
    ?arch incose:implementsFunction ?element .
    :MySystem incose:hasArchitecture ?arch .
}
```

6.3 Find Processes Applied During Development

```
PREFIX incose: <https://ontologies.incose.org/sebok/core/>

SELECT ?process ?artifact
WHERE {
    ?process incose:appliedDuringStage incose:DevelopmentStage ;
        incose:producesArtifact ?artifact .
}
ORDER BY ?process
```

7. Alignment with Standards

7.1 ISO/IEC/IEEE 15288:2023 Mapping

SEBoK/INCOSE Concept	ISO 15288 Concept
LifeCycleProcess	Life Cycle Process
TechnicalManagementProcess	Technical Management Processes
SystemDefinitionProcess	Technical Processes (Stages 1-4)
SystemRealizationProcess	Technical Processes (Implementation)
SystemVerification	Verification Process
SystemValidation	Validation Process
ConfigurationManagement	Configuration Management Process
RiskManagement	Risk Management Process

7.2 INCOSE Handbook v5.0 Alignment

The ontology directly implements concepts from:

- Chapter 2: System Life Cycle Processes
- Chapter 3: Technical Processes
- Chapter 4: Technical Management Processes
- Chapter 5: Agreement Processes (via Stakeholder)

All process definitions align with INCOSE Handbook terminology and decomposition.

7.3 ISO/IEC/IEEE 42010 (Architecture)

Architecture concepts align with ISO 42010:

- incose:Architecture aligns with architecture concept
- incose:hasArchitecture represents architecture description
- Architecture types (Functional, Logical, Physical) represent viewpoints
- Architecture frameworks can be modeled as specializations

Best Practices

Use Existing Relationships First

Before creating new properties, check if existing ones can be specialized. This maintains semantic consistency and enables better reasoning.

Maintain Traceability

Always use `incose:tracesTo` and derivation relationships. This enables impact analysis and change management throughout the system life cycle.

Document Rationale

Use `incose:rationale` to explain design decisions. This preserves knowledge and supports future maintenance and evolution.

Version Control

Track versions of artifacts using `incose:version` and `incose:versionOf`. This enables configuration management and historical analysis.

Link to Sources

Use `dct:source` to reference authoritative documents. This maintains provenance and enables verification against standards.

References

8. 1. SEBoK v2.13 (2025) - Systems Engineering Body of Knowledge
9. 2. ISO/IEC/IEEE 15288:2023 - Systems and software engineering — System life cycle processes
10. 3. ISO/IEC/IEEE 24748-1:2024 - Systems and software engineering — Life cycle management
11. 4. INCOSE Systems Engineering Handbook v5.0 (2023)
12. 5. ISO/IEC/IEEE 42010:2011 - Systems and software engineering — Architecture description

Contact and Contribution

Maintained by: INCOSE SEBoK Community

Version: 1.0

Last Updated: February 2025

License: Creative Commons Attribution-ShareAlike 4.0 International

For questions, suggestions, or contributions, please contact the SEBoK editorial team or submit issues through the INCOSE community portal.